Deep Q-Network methods to play board game (Ludo)

Sree Nirmillo Biswash Tushar, Anik Muhury
* Department of Electrical Engineering and Computer Science,
University of Tennessee, Knoxville.

***Abstract*—In this project, the agent has been trained using Q-learning to win a Pong game against computer.**
***Index Terms*—Q-Learning**

I. Introduction

Reinforcement learning (RL) is a machine learning approach that allows an artificial agent to learn on its own. An agent interacts with its environment and gets rewarded for correct actions. During the training process, the agent learns to perform the task correctly by trial and error. It interacts with the environment by making decisions based on observations it receives. Based on these observations, it predicts which actions will lead to the best possible outcomes. Based on this feedback, it adjusts its actions and tries to achieve the best possible outcomes again. The agent repeats this process until the task is completed successfully or the environment changes in a way that makes the task difficult. RL is famous for being effective for playing board games but much work was not done on the game Ludo. Therefore, The goal of this project is to better understand how these systems are developed and operated by a building an intelligent player for the board game Ludo.

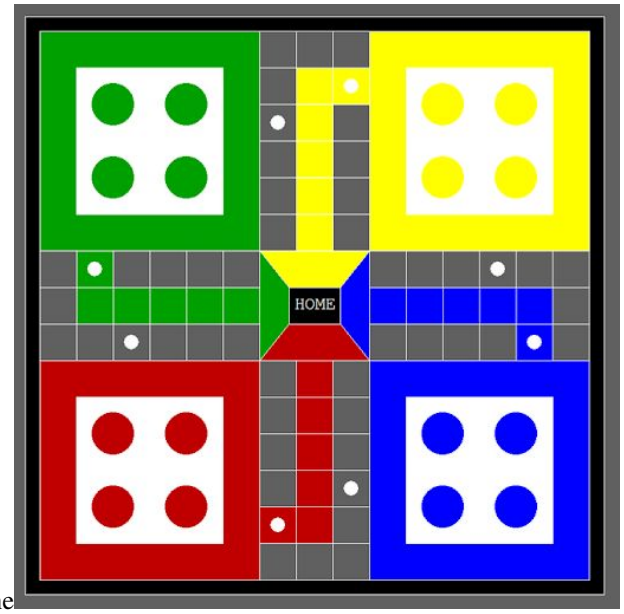


Fig. 1: Ludo boar d with pieces

Ludo is a square board game with four distinct colored bases in each corner, each split into three adjacent columns of eight squares. The start position is the first colored spot outside of each base. A path circles the board clockwise, returning to a path of the same color as the base, and then to the home column, which goes to the central home triangle. There are four distinct colored sets of playing buttons, each with its own matching foundation. Each set has four playing buttons. It can be varied from one to four buttons based on game setup. Figure-1 displays the board and pieces of a Ludo game.The accompanying die controls the movement of the buttons. The game can be played between 2 to 4 people, and there are options for becoming a partner as well. The game begins with their respective buttons being placed on their bases. Each player takes turns rolling the die, and the player with the highest roll gets to go first. The players on the left take turns moving clockwise. Each player's turn begins with a dice roll to decide a move. In order to win, a player must move all buttons once clockwise around the board, up their home column, and into their home triangle. The player is not permitted to make any further movements until at least one button is in play. If a player has a button or buttons in play, they can move any of their buttons 1 to 6 spaces down the route based on the number rolled. When a six is rolled, the player receives an additional roll following his move. For a 6, the player will have the option of moving a button from his base to the start position or moving a button that is already in play. If a six is rolled three times in a row, the player's turn is forfeited. If a player's button meets up with an opponent's button, the opponent's button is returned to their base, where he must roll a six to move it out onto the starting square again. When a button lands on the home triangle, the voyage of that button is complete. The winner is the first player whose all buttons complete their journeys.

II. Previous Study

The application of reinforcement learning strategies in board games like Ludo is a growing trend. In (G. F. Matthews and K. Rasheed, "Temporal Difference Learning for Nondeterministic Board Games,"), they use TD learning with eligibility traces where their gamer uses a similar strategy to TD-Gammon, analyzing the future board locations and selecting the best one. In their effort, they included a heuristic player to serve

as an expert and an assessment opponent for the RL-based player. Under this research (TD() and Q-Learning Based Ludo Players Majed Alhajry, Faisal Alvi, Member, IEEE and Moataz Ahmed), two RL-based Ludo players are developed: a TD player that assesses potential board situations and a Q-Learning player that chooses the optimal fundamental approach for each condition. By learning a strategy, both their TD-based player and Q-Learning Player outperform the expert player. Q-learning based Ludo game that is being reviewed here (YARAL: Yet Another Reinforcement learning Approach to Ludo) was built with an aggressive playing style in mind, with players receiving significant rewards for plays that make a lot of progress and that send their opponents home, while receiving little to no reward for inefficient moves. In that research, the effects of action and state space design, as well as other learning characteristics, were investigated. All of the previous RL-based approaches for playing the Ludo game used the tabular learning approach and there is also room to improve the formulation of the problem by changing the state representation, reward distribution, changing the learning algorithm, etc.

III. Domain Problem

In this project, we are trying to modify the existing environment available and implement a Deep Q-learning algorithm to play the Ludo game. In order to do additional research for our project, (YARAL) this gaming environment was modified. In the Ludo game, a player must be able to choose the best course of action in any situation that may arise during the game in order to advance toward the objective. State-action pairings are how this is defined for a Q-learning approach. A state space for the player's pieces must be established in order to find every potential pair. Originally in a Ludo board, there are 60 spots where a button can be. The position number actually depends on the origin of that button. The observation changes if the player or the color of the button changes. For the YARAL environment, these position numbers are changed into an encoded form to reduce the number of states. It actually sees the position of a piece and according to its surroundings, it gives a zone name. So, they actually pre-process the environment. With this method, there are only four possible states for each piece in the game which are Home, Goal Zone, Safe, and Danger. When a piece is in its "home state," it is not in play and may only be moved when a six is rolled. When a piece is in one of the final six spaces prior to the goal, it is in the goal zone. The piece is considered to be in a safe state if it is either on a globe tile or not within six tiles of a piece of the opponent in which case the next roll could send it home. The danger state is indicated when a piece is on a colored globe tile at an opponent's home or is within striking distance of an enemy piece. There are some special spots on the board, they are stars and globe. If the dice is 3 or 5 and the button is at a star location, the button can jump to the next star location. If a button is a globe tile, then it can not be killed by an opponent player. One special case is that if two buttons of a player is on the same tile location, then it can not be killed by the other player

Ten different movements can be used to define the action space. The standard playing moves specified by the game's regulations are open, normal, goal, star, globe, protect, and kill. To affect the priority of moves close to the goal, the overshoot goal and goal zone moves have been included as classification moves. The undesirable action that takes place when a piece is in the goal area but the dice roll is higher than the number of tiles required to accomplish the goal is specifically utilized as the overshoot goal move. This can be problematic because, in general, moving toward the objective area is preferable, whereas overshooting results in the piece moving further away from the target. Last but not least, the null action serves as a stand-in when the player has no active pieces to move.

As an aggressive approach was taken, actions like "goal," "star," and "kill" were rewarded most generously. By utilizing these values, the player is urged to make decisions that will advance the game the most by using stars to move lots of tiles and guiding pieces toward the objective. The "kill" action is also recommended to be used by the player to stop the opponents from advancing. By using the "globe," "protect," and "goal zone" actions, the other rewards place a somewhat lower focus on getting a piece to safety. The "normal" and "overshoot" actions get the lowest rewards because they are both regarded as undesirable moves. In conclusion, the "open" action does result in a modest reward; nonetheless, if any pieces are still in the home after rolling a 6, the "open" action is chosen. Based on prior experience with the game, this choice lessens the possibility that the player will have no active pieces and so waste turns. We included some other rewards for winning and losing the game. The agent is given a +100 reward for winning the game and a -100 reward for losing the game. Table 1 displays the rewards for each action.

Table 1: Action rewards

Action	*Reward*
Open	0.25
Normal	0.0001
Goal	0.9
Star	0.5
Globe	0.4
Protect	0.2
Kill	0.5
Overshoot	0
Goal Zone	0.4
Winning	100
losing	-100

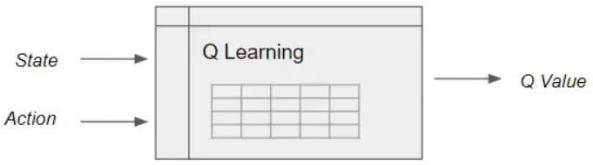


Fig. 2: Q-table of State-Action values using Q-learning

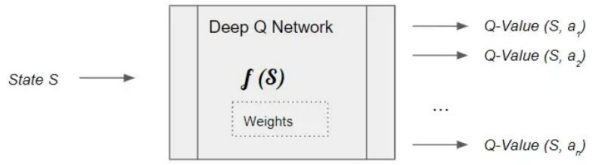


Fig. 3: Deep Q Network for estimating this Q function.

IV. Reinforcement Learning Methods

Deep reinforcement learning (DRL), which is anticipated to be a foundational technology in the future, has attracted the attention of numerous AI research facilities in recent years. In this study, we developed Deep Q-Network (DQN), the first deep reinforcement learning technique to address Q-learning drawbacks. Q Learning generates a Q-table of State-Action values that translates state and action pairs to Q-values which is demonstrated in Figure-2. However, in a realistic situation, there can be a significant number of states, making the construction of a table computationally challenging. In order to get around this restriction, we utilize a Q-function rather than a Q-table, which yields the identical state-action pair mapping to a Q value. We can estimate this Q function using a neural network that we refer to as a Deep Q Network because neural networks are efficient at simulating complex functions. This function converts one state into the Q values of all possible actions from that state. Figure-3 displays the core structure of DQN. It uses the ϵ-greedy policy to investigate the environment after starting with arbitrary Q-value estimates. In order to improve its Q-value estimates, it fundamentally employs the same concept of dual actions, such as, a current action for a current Q-value as well as a target action for a target Q-value. The Q network as well as the Target networks, along with an element referred to as Experience Replay, make up the DQN architecture. The agent that is trained to generate the ideal state-action value is the Q network. To produce data for the Q Network's training, Experience Replay engages with the environment. The Target network is the same as the Q network.

DQN can be expressed as follows to approximately represent the optimal action-value function.

$Q^*(s,a) =$
$\max_\pi \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots \mid s_t = s, a_t = a, \pi\right]$

V. First DQN Implementation

In this section we formulated the model following the structure of YAREL. That environment can be modeled in the following way.

States: There are four states for each piece of the button as described previously.

Action: There are ten actions possible as described before.

Reward: The reward structure follow Table 1.
Transition probabilities: As we don't have any exact model of the problem the transition probabilities will be inherently calculated by the DQN algorithm.

The above mention structure follows the structure of the Markov Decision Process (MDP) because an agent observes the state and takes action based on the state and transitions to a new state by achieving a certain amount of reward.
To use that in the format of DQN we coded the states using one-hat encoding. There are 16 possible positions for 4 buttons of the player. The first button could be in the home, danger zone, goal zone, and safe zone. so, for 4 buttons there are 16 locations. We input that into the neural network and used multiple linear layers with relu activation. the output of the neural network has 14 nodes. The first 4 nodes give a decision about which button to move in this turn and the next 10 nodes give a decision about the action that needs to be taken. We use argmax() for selecting the best action and the best piece to move. However, if there is no piece to move, for example, all the pieces are in the home zone, then there will be no action. If the network chooses a piece that is not present among the active pieces then a random piece will be selected among the active pieces and a random action will also be taken. we used the epsilon-greedy approach for action selection and followed the common structure of DQN in this section.

A. Results with the primary setup

To see the performance of the agent we run a simple version of the game for 10000 training and 1000 testing episodes. Without any hyperparameter tuning with a two-layer neural network with 64 neurons in each layer. The initial result during the training suggests that the average reward does not increase that much and the percentage of wins the agent sees is also around 27%. To find a better result we tried many versions of the game to test and see the performance. To see the effect of different hyperparameters on the average reward, we run 10000 episodes for training and 1000 episodes for testing. The average reward is calculated after every 100 episodes. Figure 4 shows the average return during training for different gamma values. It can be seen that gamma=0.99 is not working well and all other gamma values are fine for this case. From the nature of the ludo game, we can also verify this phenomenon. The recent reward is not always very much important for this game. While taring we saw that gamma=0.7 gives the highest percentage of the win (30%), hence, we selected that.

Figure 5 shows the effect of the learning rate on the average return. It can be seen that neither a higher nor a smaller learning rate works so well for this problem. However, 0.1 is a good selection as a learning rate because we get a stable average

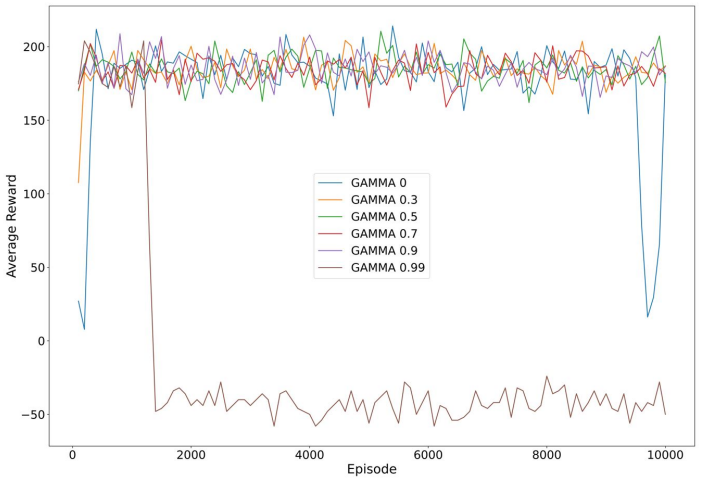


Fig. 4: Average Return with different discount factor

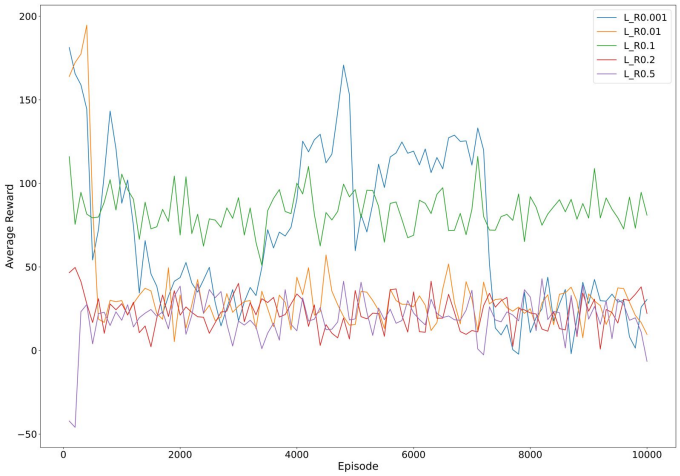


Fig. 5: Average Return with different learning rate

return and the highest percentage of wins (32%) for this one. Therefore we selected that. The effect of different kinds of state representation can be seen in Figure 5. From the figure, it is evident that if we represent states as coded representations, the average return will be good. So, we selected the coded representation. The depth of the network used to learn the Q-values is also an important parameter to tune. Four layer structure is used and from Figure 7 we can see that the medium-deep layer structure actually works really well for this problem. So, we selected [64, 64, 64, 64] as the layers of our DQN network.

VI. Code design and data structures

- Episodes: This methods runs one episode. First of all, the state is returned using the Pong class. The returned state is 6 continuous state, y1: the midpoint of the paddle of the agent, y2:The midpoint of the paddle of the computer, xball: the x position of the ball, yball: the y position of the ball, ballHDirection: horizontal projection of the ball, ballVDirection: vertical projection of the ball. Among them, only y1, xball, yball were taken. In addition to that, another state is taken as abs(y1-yball) since the difference of height between ball and paddle is important to hit the ball. All of those 6 states are in the range of (0,300) range. Therfore, the values of y1,xball,yball are discretized to (0,7). Therefore, the state space becomes (8,8,8,8) and action value method has a size of (8,8,8,8,3). After getting and converting a state, an action is taken using epsilon greedy method. This will give us a reward and the next state. Finally, Q(state,action) will be updated using equation 1:
$Q(s,a) = Q(s,a) + a * (r + gamma * \max_a Q(s',a) - Q(s,a))$.
- State Conversion: This method is used to convert continuous range (0,300) to discrete (0,7).
- RunLearning: This method was used to train and test the model after after c episodes. Firstly, the model is trained c episodes and the online total winning and average return is stored in an array to plot in a graph later. Then, after c episodes, the trained Q is to used evaluate the model offline. The offline total winning and the average return is again stored in a variable. Lastly, online average return

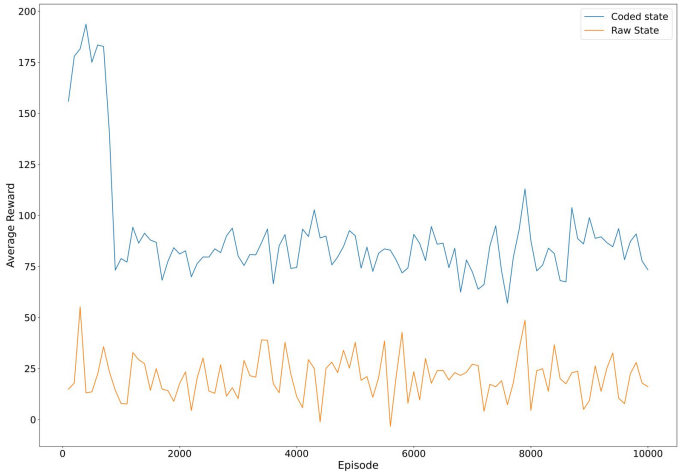


Fig. 6: Average Return with different state representation

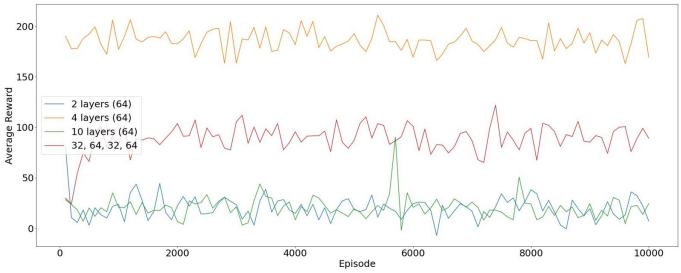


Fig. 7: Average Return with different layer structure

```
Average Return: -97.546 and win 0
Average Return: -97.112 and win 0
Average Return: -97.222 and win 0
Average Return: -98.914 and win 0
Average Return: -98.502 and win 0
Average Return: -97.884 and win 0
Average Return: -99.716 and win 0
Average Return: -98.42 and win 0
Average Return: -97.948 and win 0
Average Return: -97.702 and win 0
Average Return: -98.262 and win 0
Average Return: -99.136 and win 0
Average Return: -98.358 and win 0
Average Return: -99.458 and win 0
Average Return: -97.698 and win 0
Average Return: -99.262 and win 0
Average Return: -99.57 and win 0
Training take 899.6227219104767
```

Fig. 8: Screenshot

and the offline average return is plotted in a graph. In addition to that, the total winning for online and offline approach is plotted in another graph.

- Test: This methods takes Q and the agent play a 500 games with the Q offline to find out the total number of winning and average return. This method is used in RunLearning.

VII. Result

A. Result Analysis

How much does the agent wins if the quantization is reduced from (8,8) grid to (4,4) grid?
If the quantization is reduced, the average return and the winning will reduce. But, in this case, how much the return and the total win reduces was intended to answer here. The figure 8 shows the online learning average returns and the training time. It is obvious that the training time reduces but the agent could not win a single game. Figure 9 shows that the agent hit 0-3 times and lose online and offline. Therefore, the figure 10 shows the flat the winning curve for both online and offline.

What is the effect of adding an discount factor of 0.9?
Using the discount factor reduces the average return (figure 12) and the total number of wins (figure 13) compared to the original result (figure **??**, figure **??**). The reason might be the reward is discounted and getting smaller which is affecting the Q(s,a) update.

VIII. Conclusion

In this project, the states are discretized and shaped into an 8X8 grid, which affects the outcome. We also discover from our investigation that decreasing the grid always has a

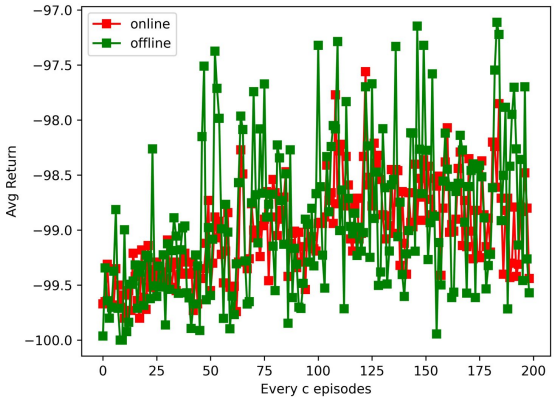


Fig. 9: Average Return

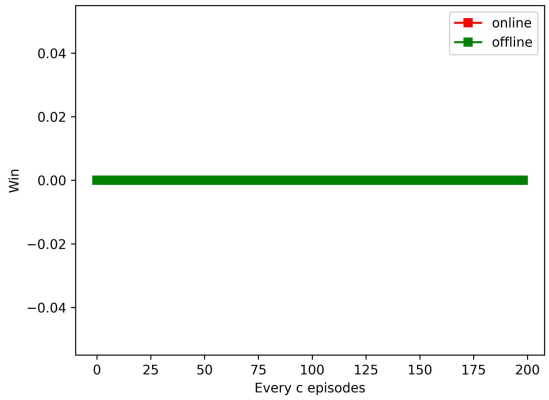


Fig. 10: Win

```
Average Return: -74.018 and win 43
Average Return: -74.64 and win 40
Average Return: -48.666 and win 84
Average Return: -29.516 and win 124
Average Return: -75.768 and win 39
Average Return: -44.418 and win 96
Average Return: -26.172 and win 135
Average Return: -34.578 and win 119
Average Return: -29.794 and win 132
Average Return: -83.712 and win 20
Training take 4887.170882701874
```

Fig. 11: Screenshot

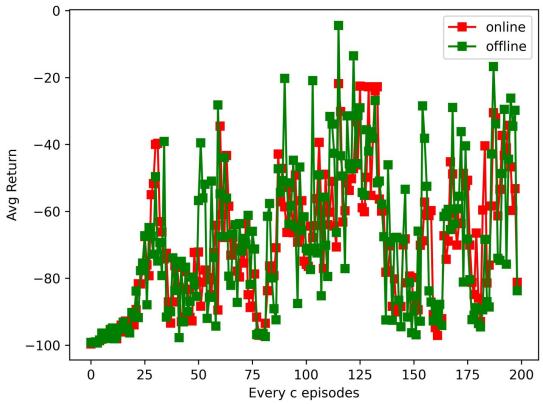


Fig. 12: Average Return

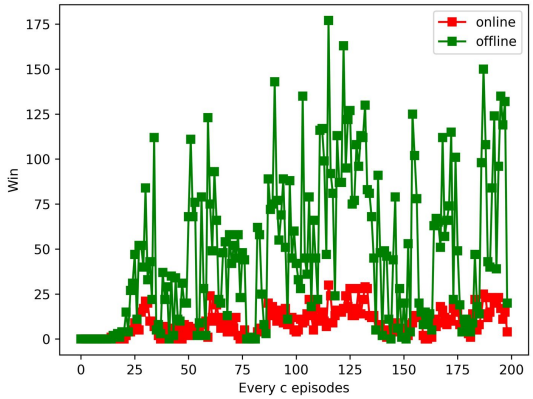


Fig. 13: Win

negative impact on learning. When given the opportunity to train over longer periods of time, the agent always learns more effectively, which enables it to update Q(s, a) from updated returns. Here, we select the unit discount factor, but when we opt for discounted discount factor, it has a negative impact on the return and eventually has a bad impact on agent performance.